



Project Report
Project 07

NIDS on Raspberry Pi 4
Network Intrusion Detection
Implemented on Raspberry Pi 4

Prepared by:

Hafsa AKZOUN
Farah ABERKANE
Anishka SELVANATHAN
Abdoul-Hakim SACKO

Mentors:

Mrs. Anne Fladenmuller
Mr. Giuliano Fittipaldi

Report submitted on 10/05/2026

Contents

1	General Project Context	1
1.1	Introduction	1
1.2	Project Presentation	1
1.3	Project Management Approach	2
2	Literature Review and Dataset Analysis	4
2.1	Introduction	4
2.2	Network Intrusion Detection Systems	4
2.3	Types of Network Attacks	5
2.4	Machine Learning in Cybersecurity	5
2.5	Dataset Analysis: UNSW-NB15	6
3	System Design	11
3.1	Introduction	11
3.2	Overall System Architecture	11
3.3	Hardware and Deployment Platform	12
3.4	Software Environment	13
4	Two-Level Model Development, Training, and Evaluation	14
4.1	Introduction	14
4.2	Level 1: Binary Detection Model	14
4.3	Level 2: Attack Category Classification Model	17
5	Implementation on Raspberry Pi 4	20
5.1	Introduction	20
5.2	Environment Setup on Raspberry Pi	20
5.3	Deploying the Selected Models	21
5.4	Testing and Performance Evaluation	22
5.5	Global Performance of the Two-Level Approach	23
6	Federated Learning	24
6.1	Introduction	24

6.2	Motivation for Federated Learning in Intrusion Detection	24
6.3	Federated Learning Architecture	24
6.4	Federated Training Strategy for Level 1	25
6.5	Discussion and Limitations	25
6.6	Conclusion	26
7	General Conclusion	27
7.1	Summary of Achievements	27
7.2	Challenges Encountered	27
7.3	Limitations and Improvement Perspectives	27

Chapter 1: General Project Context

1.1 Introduction

The massive growth of computer networks and the diversity of applications have created complex cybersecurity challenges. Intrusions, defined as a set of events compromising the availability, authority, confidentiality, or integrity of systems, require advanced protection mechanisms that exceed the capabilities of traditional firewalls. This project is part of this dynamic, aiming to develop a Network Intrusion Detection System (NIDS) capable of analyzing traffic flows in depth to identify modern threats.

1.2 Project Presentation

1.2.1 Project Context

The development of an effective NIDS relies on the use of a comprehensive dataset that reflects both normal behaviors and abnormal network activities. Historically, research has relied on benchmarks such as KDDCUP 99 and NSLKDD. However, these datasets are now considered obsolete because they do not reflect the current threat environment, particularly "low footprint" attacks.

Our project uses the UNSW-NB15 dataset, created by the Australian Centre for Cyber Security (ACCS). This dataset was generated via a testbed using the IXIA PerfectStorm tool to simulate a hybrid traffic environment, mixing modern real activities with contemporary malicious behaviors. The deployment of our solution focuses on the use of a Raspberry Pi microcomputer, transforming this resource-constrained device into an active security probe.

1.2.2 Problem Statement

The evaluation and effectiveness of NIDS face several major obstacles identified in the academic literature:

- **Inadequacy of classic datasets:** Older datasets like KDDCUP 99 contain numerous redundant records that bias detection results toward the most frequent patterns.
- **Absence of modern threats:** Historical benchmarks are not representative of current attack techniques, which limits the relevance of systems trained on these datasets.
- **Hardware constraints:** Edge devices such as the Raspberry Pi 4 have limited CPU and memory resources compared to standard machines, which constrains the complexity of deployable models.

The central problem of this project is therefore to design a system capable of maintaining a high level of detection for modern threats (e.g., Fuzzers, Backdoors, DoS) for a restricted hardware architecture.

1.2.3 Project Objectives

To address the identified challenges, the operational objectives of this project are as follows:

- **Network traffic analysis:** Utilize the 49 features extracted from the UNSW-NB15 dataset, including flow, content, and temporal characteristics, for precise classification.
- **Hybrid Detection on Raspberry Pi:** Design a lightweight and efficient NIDS capable of classifying nine families of attacks in real-time on resource-constrained hardware.
- **Comparison and Performance:** Validate the system's effectiveness based on the richness of the UNSW-NB15 dataset, which offers a more diverse and modern representation than previous standards.

1.3 Project Management Approach

To organize the project work, we adopted an Agile approach based on the Scrum framework. This helped us divide the project into clear phases, follow progress regularly, and adapt the work when difficulties appeared.

Task Distribution and Project Progress

This table presents the main project tasks managed during the development phase.

Task ID	Work	Assignee	Status
PN-1	Project Planning and Setup	Team	DONE
PN-2	Dataset Understanding	Abdoul-hakim SACKO	DONE
PN-3	Data Preprocessing Pipeline	Anishka Selvanathan	DONE
PN-4	Level 1 Model: Normal vs Attack	Farah Aberkane	DONE
PN-5	Level 2 Model: Attack Category Classification	Hafsa Akzoun	DONE
PN-6	Two-Level Detection Pipeline	Team	DONE
PN-7	Raspberry Pi 4 Deployment	Farah Aberkane & Abdoul-hakim SACKO	DONE
PN-8	Federated Learning Module	Hafsa Akzoun & Abdoul-hakim SACKO	DONE
PN-9	Testing and Validation	Abdoul-hakim SACKO	DONE
PN-10	Prepare Final Deliverables	Team	DONE

Table 1.1: Project task distribution and progress

Planning and Timeline

The project was divided into several phases: literature review, data set analysis, model development, Raspberry Pi deployment, and Federated Learning. Each phase was divided into

smaller tasks to make progress easier to track.

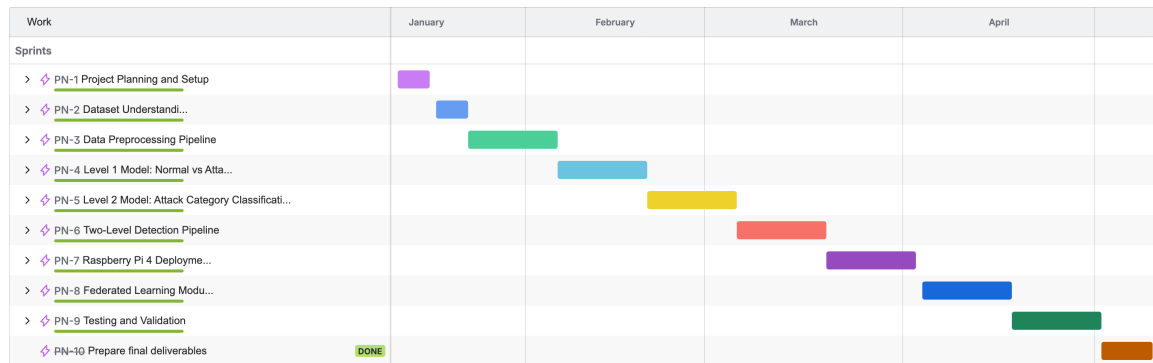


Figure 1.1: Gantt-style overview of the project timeline

Chapter 2: Literature Review and Dataset Analysis

2.1 Introduction

This chapter presents the technical background of the project. It introduces Network Intrusion Detection Systems (NIDS), their main principles, challenges, and detection approaches. It also describes the types of attacks in the UNSW-NB15 dataset and the role of machine learning in cybersecurity.

The chapter then presents the dataset and a basic data analysis, including the data structure, target distribution, feature relationships, and class imbalance.

2.2 Network Intrusion Detection Systems

2.2.1 Overview of NIDS

A Network Intrusion Detection System (NIDS) is a security tool that monitors network traffic to detect suspicious activities that could threaten the confidentiality, integrity, or availability of systems. NIDS passively observe traffic and analyze packets; they are usually placed at entry and exit points, central nodes, or near critical servers.

The performance of a NIDS depends on several factors, including the quality of its detection rules or models, its ability to handle large amounts of traffic, and its accuracy. In particular, it must reduce false positives and false negatives.

2.2.2 Signature-Based Detection

Signature-based detection works by comparing network activity to a database of known attack signatures.

The main advantage of this approach is that when a signature is well defined, it can detect attacks precisely with very few false positives. However, this approach cannot detect attacks that are not already in the database.

2.2.3 Anomaly-Based Detection

Anomaly-based detection learns what normal network behavior looks like and detects any unusual activity as a potential threat.

Since this method does not depend on known signatures, it can detect new threats. However, it is more difficult to implement because defining “normal” behavior is challenging, as network traffic constantly changes depending on users, applications, and other factors.

2.2.4 Hybrid Detection Approaches

This approach combines both signature-based and anomaly-based detection methods; signature-based detection is used to quickly identify known attacks, while anomaly-based detection is

used to detect unknown threats by analyzing unusual behavior.

This combination improves overall detection performance, allowing the system to be both precise for known attacks and effective against new ones

2.3 Types of Network Attacks

The UNSW-NB15 dataset contains a wide variety of attacks. The model must be able to identify several attack families, including :

- **Denial of Service (DoS):** flooding a system to make it unavailable.
- **Fuzzers:** sending invalid data to detect vulnerabilities.
- **Backdoor:** bypassing authentication to gain hidden access.
- **Exploits:** using a vulnerability to compromise a system.
- **Shellcode:** malicious code used during an exploit.
- **Worms:** self-replicating programs that spread across networks.
- **Reconnaissance:** collecting information about a target.
- **Generic:** attacks using common techniques without specific signatures.
- **Analysis:** probing activities such as scanning or spam.

The distribution of these attack categories in the UNSW-NB15 dataset is highly imbalanced, with some attack types being much more frequent than others.

2.4 Machine Learning in Cybersecurity

2.4.1 Role of Machine Learning in Intrusion Detection

In this project, machine learning is essential for improving intrusion detection. By learning from the UNSW-NB15 dataset, the model can detect abnormal behaviors, even when they do not match known attack patterns.

It also enables a two-step approach: first, a binary classification to distinguish normal from malicious traffic, then a multi-class classification to identify the type of attack. This structure improves accuracy and simplifies the task.

In addition, machine learning can handle large volumes of data and capture complex relationships between features, which would be difficult to model manually.

2.4.2 Supervised Learning Approaches

Supervised Learning uses labelled data to learn how to make predictions. Each data example is associated with a known output, which allows the model to learn the relationship between input features and the correct class.

In this project, supervised learning is used to train the intrusion detection system on the UNSW-NB15 dataset. The model learns to classify network traffic as normal or malicious, and to identify the type of attack.

2.5 Dataset Analysis: UNSW-NB15

2.5.1 Overview of the UNSW-NB15 Dataset

The UNSW-NB15 dataset (University of New South Wales – Network Benchmark 15) is a benchmark dataset for network intrusion detection, containing a mixture of normal network traffic and malicious traffic representing 9 attack categories: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms.

It is provided in CSV format and divided into two subsets:

- `UNSW_NB15_training-set.csv` : a training set containing 175,341 network traffic records, used to train the machine learning models.
- `UNSW_NB15_testing-set.csv` : a testing set containing 82,332 network traffic records, used to evaluate model performance.

Both subsets share the same 45 features and are complete, no missing values are present, allowing the pipeline to proceed directly to the preprocessing stage.

2.5.2 Exploratory Data Analysis

The exploratory analysis reveals that the majority of the dataset's features are numerical, with the exception of four categorical features that will require encoding before they can be used by machine learning models.

The 45 features are distributed as follows:

- **41 numerical features**: 30 integers (`int64`) and 11 decimals (`float64`), representing network metrics such as byte volumes, packet counters, connection durations, and data rates.
- **4 categorical features** (object):
 - `proto` (network protocol) : 133 unique values (TCP, UDP, ICMP...)
 - `service` (network service) : 13 values (HTTP, FTP, DNS...)
 - `state` (connection state) : 9 values (FIN, INT, CON...)
 - `attack_cat` (attack category) : 10 values. This is the target variable for multi-class classification.

These categorical features exhibit very different cardinalities: `state` and `service` have few distinct values and are therefore well-suited for one-hot encoding, whereas `proto`, with its 133 unique values, requires a more considered encoding strategy to avoid a dimensionality explosion. The features `label` and `is_sm_ips_ports`, already binary (0 or 1), require no transformation.

2.5.3 Target Variable Distribution

Binary Classification Label

The target variable `label` is binary: 0 for normal traffic and 1 for attack traffic.

Its distribution across both subsets is as follows:

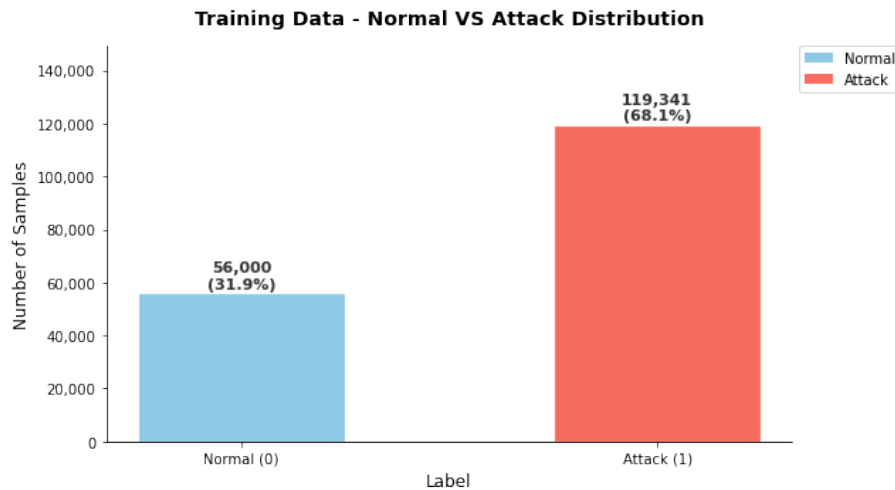


Figure 2.1: *Label Distribution in Training Data.*

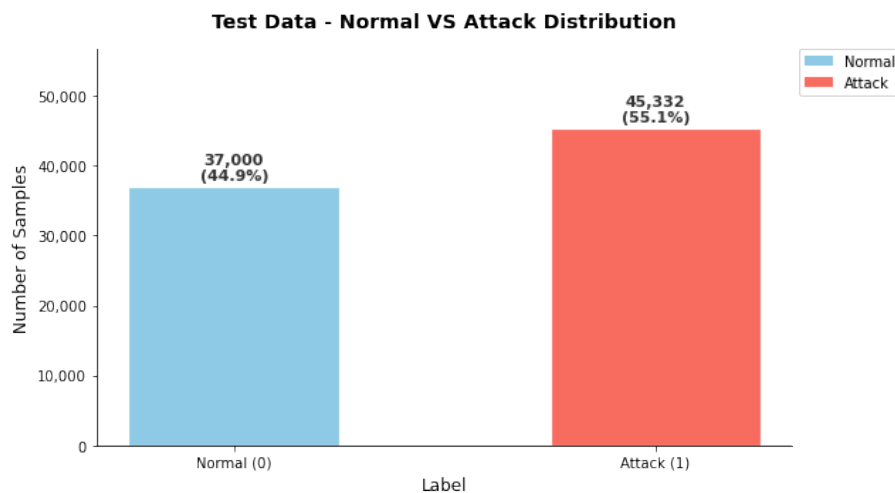


Figure 2.2: *Label Distribution in Testing Data.*

Attack Category Distribution

The `attack_cat` variable identifies the precise category of each network record among 10 classes. Its distribution across both subsets reveals a pronounced multi-class imbalance:

**Distribution of Attack Categories
(Training Set)**

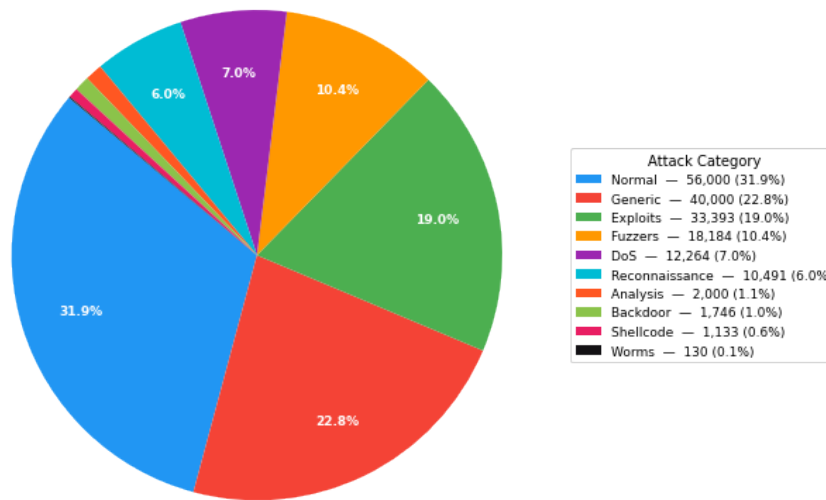


Figure 2.3: Attack Distribution in Training Data.

**Distribution of Attack Categories
(Testing Set)**

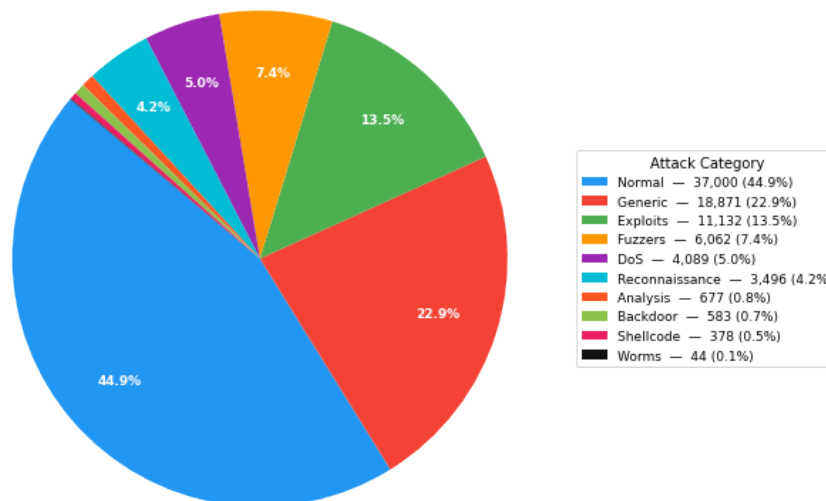


Figure 2.4: Attack Distribution in Testing Data.

2.5.4 Feature Correlation Analysis

The correlation matrix was computed on the 41 numerical features to identify strongly correlated variables:

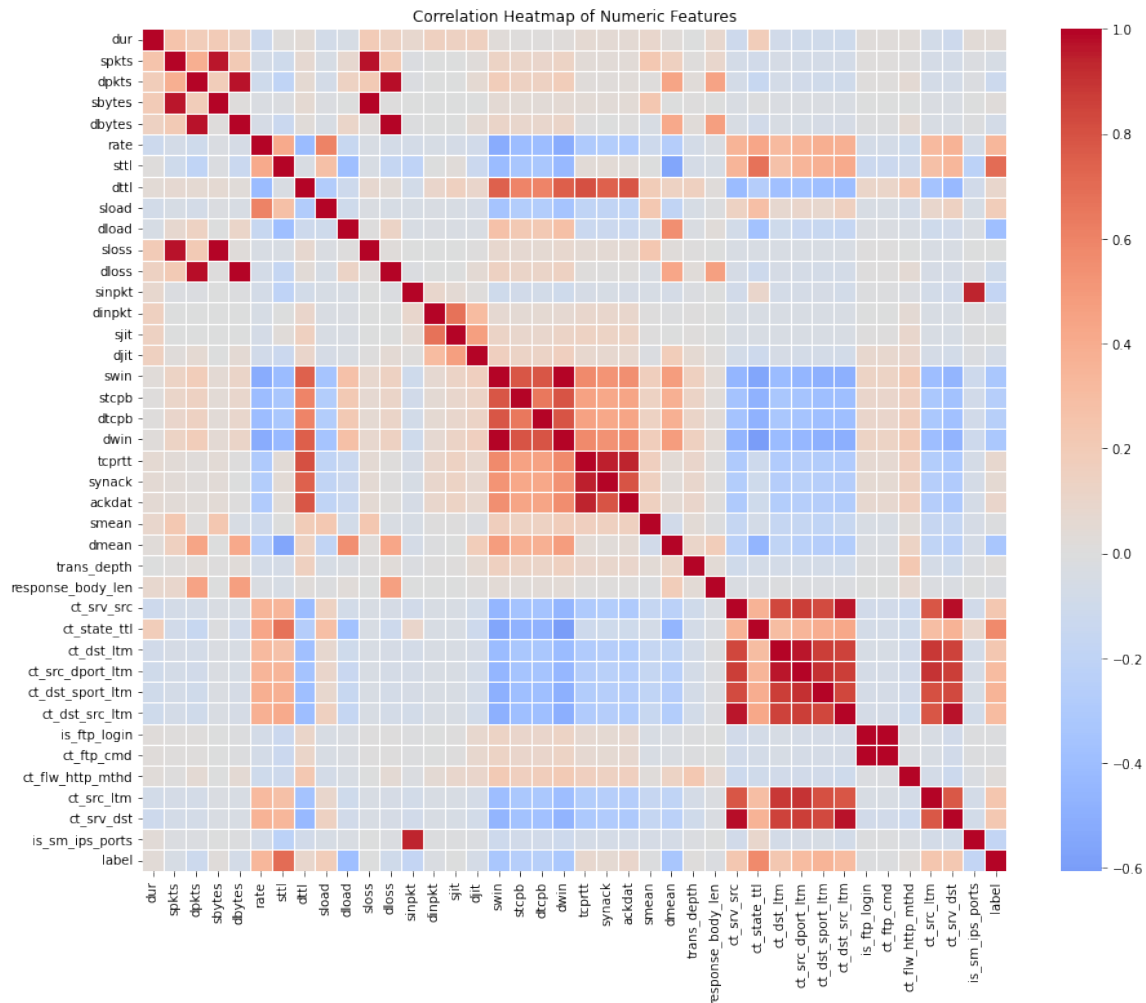


Figure 2.5: Correlation Heatmap of Numeric Features.

Inter-feature Correlation

- spkts, sbytes and sloss : a high number of sent packets naturally implies a larger data volume and more packet losses, explaining the strong correlation among these three features.
- dpkts, dbytes and dloss : the same reasoning applies on the destination side.
- swin and dwin : the source and destination TCP window sizes are closely linked, reflecting the symmetric behaviour of a TCP session.
- tcprtt, synack and ackdat : tcprtt represents the total round-trip time, which by definition encompasses synack and ackdat, hence their high correlation.
- ct_srv_src and ct_srv_dst : these two features measure activity between source/destination pairs with partially overlapping information.
- is_ftp_login and ct_ftp_cmd : perfectly correlated, both features signal active FTP activity.

Correlation Between Features and Labels

- `sttl` and `ct_state_ttl` emerge as the most discriminating indicators, reflecting TTL value variations and state transitions characteristic of certain attack categories.
- `ct_dst_sport_ltm`, `ct_src_dport_ltm`, `ct_dst_src_ltm`, `ct_dst_ltm`, `ct_src_ltm` : long-term connection metrics that exhibit moderate to high correlations, highlighting persistent or distributed behaviours often associated with targeted and repetitive attacks.
- `state` shows a notable correlation, confirming the importance of connection state differences, such as resets or incomplete handshakes, in distinguishing normal from malicious traffic.
- `rate` presents a moderate correlation, indicating its usefulness in identifying attacks characterised by unusual traffic spikes, such as DDoS or flood attacks.
- `ct_srv_dst`, `ct_srv_src` : service metrics that provide information on service-destination and service-source targeting patterns, often associated with the exploitation of known vulnerabilities in certain attack categories.

2.5.5 Class Imbalance Analysis

Binary Classification (`label`)

As shown in Figure 2.1, the analysis of the target variable `label` distribution reveals a significant class imbalance, with 68.06% attack traffic against 31.94% normal traffic.

The following strategies are adopted to address this imbalance:

- `class_weight='balanced'` : automatically adjusts class weights during training.
- **SMOTE** (Synthetic Minority Oversampling Technique) : generates synthetic samples for the minority class.
- Recall and F1-score as the primary evaluation metrics, rather than accuracy.

Multi-class Classification (`attack_cat`)

The imbalance is even more pronounced for attack category classification.

Certain attack categories largely dominate the dataset, accounting alone for more than 70% of observations (Normal, Generic, Exploits), while others are extremely rare, each representing less than 1% of the dataset, notably Worms, Shellcode, and Backdoor.

This severe imbalance makes multi-class classification considerably more challenging, as models tend to neglect minority classes in favour of dominant ones. This finding reinforces the need to resort to:

- **Oversampling** to increase the representation of rare classes.
- **Hierarchical classification (cascading)** by first separating binary detection (normal vs. attack), then performing fine-grained classification by category.
- **Class-weighted models** to assign greater importance to minority classes during training.

Chapter 3: System Design

3.1 Introduction

This chapter details the technical design of our Network Intrusion Detection System (NIDS). It presents the overall architecture, based on a two-level cascade detection approach, as well as the technological choices made.

3.2 Overall System Architecture

3.2.1 General Architecture of the Proposed NIDS

The proposed NIDS relies on a two-level machine learning pipeline, trained on the UNSW-NB15 dataset. Captured network traffic is first preprocessed (normalisation, categorical feature encoding), then submitted to a cascade classification chain. The entire pipeline is deployed on the Raspberry Pi 4 to operate in real time.

3.2.2 Two-Level Detection Workflow (Cascade)

To address the class imbalance of the dataset, we propose a cascade classification architecture. This multi-step approach divides the problem into more balanced sub-problems, thereby improving overall detection performance.

Level 1: Normal vs. Attack Detection

The first level is a binary classifier that distinguishes normal traffic from malicious traffic. This initial separation is essential, as the “Normal” class exhibits characteristics that are distinct from all attack categories. Traffic classified as normal is immediately released; only traffic identified as an attack is passed to the next level. This model is optimised for high Recall in order to minimise false negatives.

Level 2: Attack Category Classification

Flows detected as malicious at Level 1 are forwarded to the second level, which performs multi-class classification among the 9 attack categories of the dataset. By focusing exclusively on malicious traffic, this model can learn finer decision boundaries between the different attack types, without being disturbed by legitimate traffic. A resampling mechanism is applied to compensate for the severe imbalance among attack categories, particularly the very rare classes such as Worms and Shellcode.

3.2.3 Deployment Graph

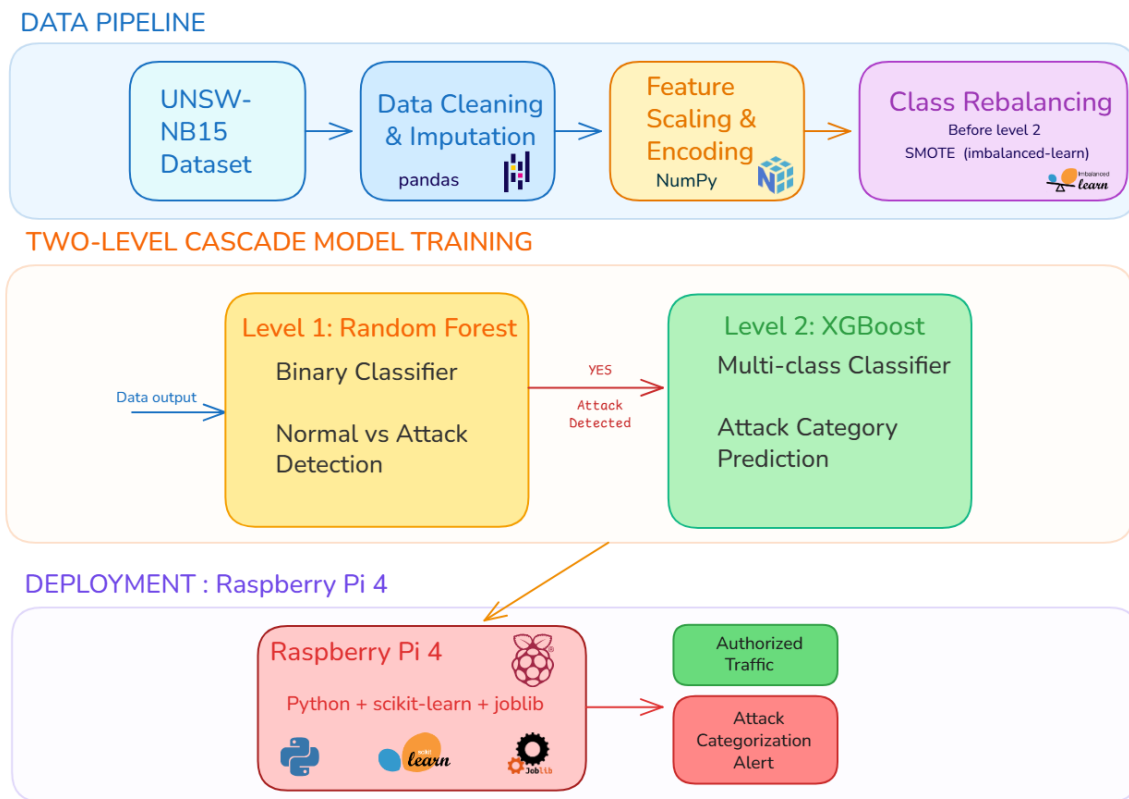


Figure 3.1: Deployment Graph.

3.3 Hardware and Deployment Platform

3.3.1 Raspberry Pi 4 Specifications

The Raspberry Pi 4 Model B constitutes the reference hardware platform for this project. Its main specifications are as follows:

- **CPU:** Quad-core Cortex-A72 64-bit SoC @ 1.5 GHz
- **RAM:** 4 GB
- **Storage:** MicroSD 32 GB
- **Connectivity:** Wi-Fi & Ethernet

3.3.2 Suitability of Raspberry Pi 4 for NIDS Deployment

The Raspberry Pi 4 presents several characteristics that make it particularly well suited for embedded NIDS deployment:

- **Dedicated computer :** it can be configured exclusively for network traffic monitoring, without interference from other processes.
- **Python / machine learning ecosystem compatibility :** trained models can be deployed and run real-time inference on captured traffic.

- **Low power consumption** : enables continuous 24/7 operation at minimal cost.
- **Remote management** : the system can send alerts to the network administrator without requiring a wired connection.

3.4 Software Environment

3.4.1 Operating System and Development Environment

Raspberry Pi OS (Raspbian, 64-bit) was retained as the operating system due to its lightweight footprint, reliability, and native compatibility with Python development tools. Models are trained on a PC, then exported and loaded directly onto the embedded platform for real-time inference.

3.4.2 Programming Language and Main Libraries

Python was chosen as the primary language due to its extensive collection of machine learning libraries and its compatibility with Raspberry Pi OS. The main libraries used cover the entire pipeline, from preprocessing to classification:

- **pandas / numpy**: data manipulation and analysis
- **matplotlib**: visualisation
- **scikit-learn**: preprocessing, classification models, metrics
- **imbalanced-learn**: class imbalance handling
- **joblib**: model serialisation and deployment
- **flwr** (Flower): Federated Learning

Chapter 4: Two-Level Model Development, Training, and Evaluation

4.1 Introduction

This chapter presents the design, implementation, training, and evaluation of the two-level intrusion detection model. The system first detects whether network traffic is normal or an attack, then identifies the type of attack from the UNSW-NB15 dataset.

4.2 Level 1: Binary Detection Model

4.2.1 Preprocessing Pipeline for Level 1

The preprocessing pipeline transforms the dataset through three main steps: categorical encoding, feature selection, and feature scaling.

First, the columns `id` and `attack_cat` are removed. The target variable is `label`, where 0 corresponds to normal traffic and 1 to an attack. As shown in Figure 2.1, the data is imbalanced. This is handled during training using the `class_weight='balanced'` parameter.

Since categorical features (`proto`, `service`, and `state`) have high cardinality, only the 10 most frequent values are kept for each feature, while all others are grouped into an "others" category. A `LabelEncoder` is then trained on the training set and applied to both training and test data.

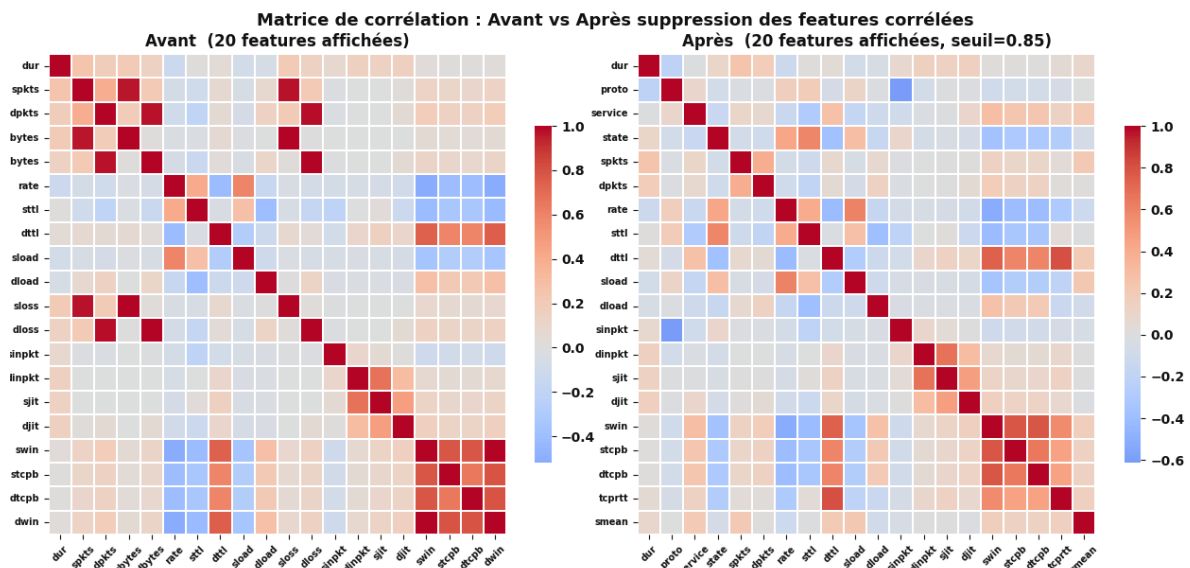


Figure 4.1: Correlation matrices before and after removing highly correlated features.

Finally, redundancy among features is reduced using correlation-based filtering. Several thresholds (0.50, 0.80, 0.85, and 0.90) were tested, and 0.85 was selected as the best balance between reducing redundancy and preserving information, reducing the set of characteristics from 47 to 28 variables. The impact of this filtering is shown in Figure 4.1

4.2.2 Model Training for Level 1

Random Forest Training

The model is set with 200 trees and a maximum depth of 15. This limits complexity and helps prevent overfitting while still keeping good performance. The parameter `class_weight='balanced'` is used to handle class imbalance by giving more importance to rare classes.

Neural Network Training

The neural network used is a Multi-Layer Perceptron (MLP). The training stops automatically if the model stops improving on the validation set.

Compared to Random Forest, the MLP is more sensitive to feature scaling, which is why the StandardScaler applied during preprocessing is essential for this model to work correctly.

ROC-Based Threshold Optimization

We observed a high number of false positives, which is problematic in a security context because it would block normal network traffic. So we needed to reduce these false alarms. For this reason, we adjusted the decision threshold using the ROC curve for both models.

The optimal threshold is selected at the point that maximises the difference between TPR and FPR.

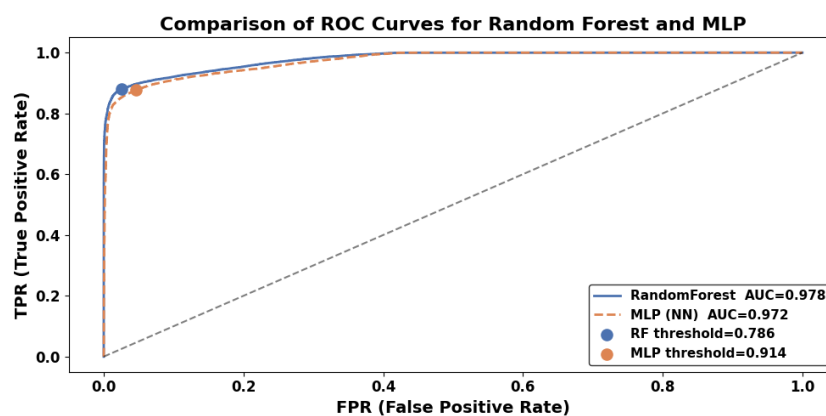


Figure 4.2: ROC Curves with Optimal Decision Thresholds for Random Forest and MLP

4.2.3 Results and Evaluation of Level 1

Binary Classification Results

The performance obtained on the test set is as follows:

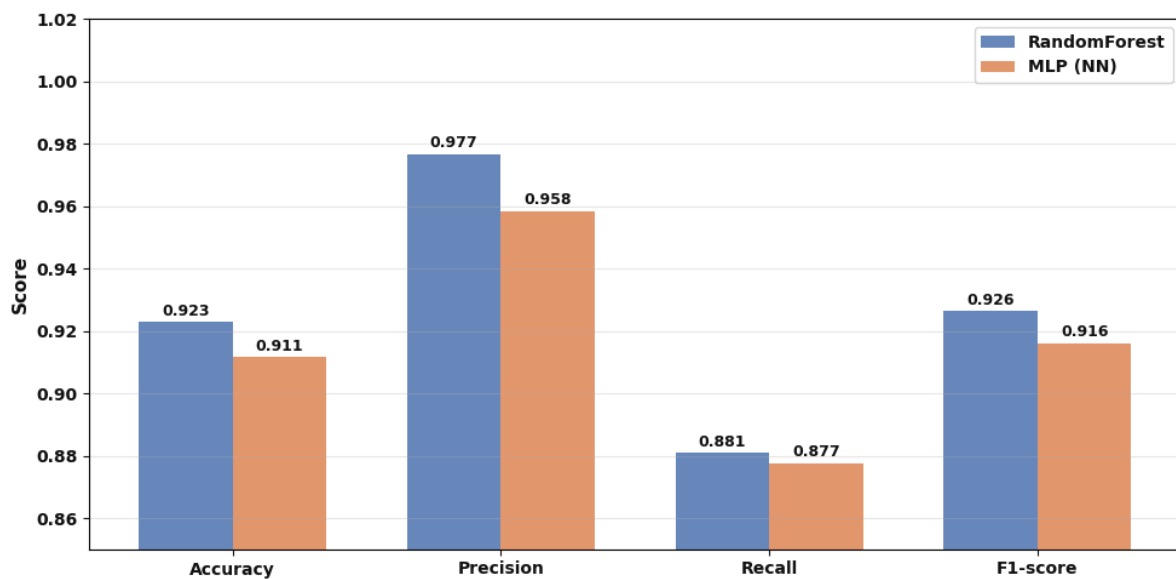


Figure 4.3: Comparison of level 1 models

Confusion Matrix Analysis

The Random Forest correctly classifies 36,045 normal connections and detects the vast majority of attacks, with 40,000 true positives. It produces 955 false positives, and 5,403 false negatives (attacks that were missed).

The Neural Network shows slightly weaker results on both fronts. It generates 1,728 false positives, nearly twice as many as the RandomForest, and 5,559 false negatives.

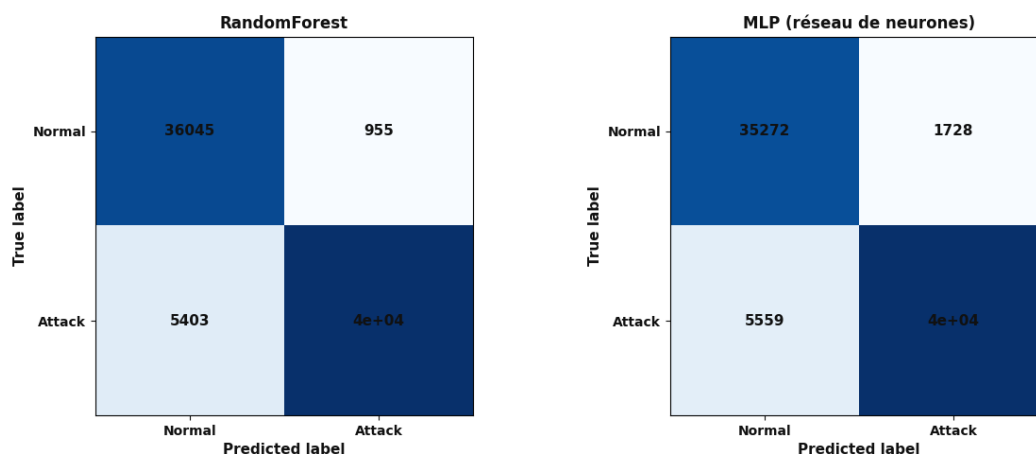


Figure 4.4: Confusion matrix .

While the number of correctly detected attacks remains comparable between the two models, the RandomForest is clearly more precise in identifying normal traffic, which is critical in a real-world deployment where false alarms can disrupt legitimate network activity.

4.3 Level 2: Attack Category Classification Model

Following binary classification, Level 2 focuses exclusively on malicious traffic and assigns each attack flow to one of nine attack categories.

After filtering for malicious flows, 119,341 training samples and 45,332 test samples remain for Level 2 analysis.

4.3.1 Level 2 Data Preparation

The Level 2 preprocessing pipeline was modified to support multi-class attack classification. The main steps included the following:

- **Attack filtering:** retained only malicious flows, identified by `label = 1`.
- **Target selection:** designated `attack_cat` as the target class for prediction.
- **Column removal:** excluded `id`, `label`, and `attack_cat` from the input features.
- **Rare category grouping:** replaced categorical values with frequencies below 0.2% with the category `OTHER`.
- **Correlation filtering:** eliminated numerical features with correlation coefficients greater than 0.90.
- **Encoding and scaling:** encoded categorical features and scaled numerical features.
- **Imbalance handling:** addressed class imbalance using `SMOTETomek` together with `SMOTENC`.

Following preprocessing, the dataset comprised 42 input features: three categorical features, one binary feature, and the remaining numerical features. Correlation filtering removed 13 numerical features, retaining 25. After encoding, the final feature space consisted of 46 processed features.

To address class imbalance, the training set size increased from 107,406 to 297,400 samples after applying `SMOTETomek` with `SMOTENC`.



Figure 4.5: Class distribution before and after `SMOTETomek` for Level 2 classification

4.3.2 Model Training and Comparison

Several machine learning models were trained and evaluated using the same preprocessing pipeline: XGBoost, Random Forest, Extra Trees, Linear SVC, and Logistic Regression. Tree-based models were used to capture non-linear relationships, while Linear SVC and Logistic Regression were included as simpler baseline models. XGBoost was trained with 1000 estimators, a learning rate of 0.05, a maximum depth of 5, and early stopping.

The models were compared using Accuracy, Balanced Accuracy, Macro F1-score, and Weighted F1-score. Since the dataset is highly imbalanced, Macro F1-score and Balanced Accuracy were considered the most relevant metrics.

Model	Accuracy	Balanced Accuracy	Macro F1	Weighted F1
XGBoost	0.7769	0.5856	0.5382	0.7991
Random Forest	0.7584	0.5513	0.5154	0.7876
Extra Trees	0.7478	0.5633	0.5011	0.7773
Linear SVC	0.6380	0.5262	0.3485	0.6842
Logistic Regression	0.6364	0.5041	0.3366	0.6801

Table 4.1: Comparison of machine learning models for Level 2 classification

XGBoost achieved the best overall performance, with the highest Accuracy, Macro F1-score, and Weighted F1-score. It was therefore selected as the final Level 2 attack classification model.

4.3.3 Final XGBoost Configuration and Results

The selected XGBoost model was then evaluated on 45,332 malicious test samples. It was trained with the `multi:softprob` objective, 1000 estimators, a learning rate of 0.05, a maximum depth of 5, and early stopping. The best validation multiclass log loss was 0.55558.

Attack Category	Precision	Recall	F1-score	Support
Analysis	0.0582	0.0871	0.0698	677
Backdoor	0.0362	0.1201	0.0556	583
DoS	0.3162	0.4285	0.3639	4089
Exploits	0.7786	0.6604	0.7147	11132
Fuzzers	0.8734	0.7667	0.8166	6062
Generic	0.9995	0.9645	0.9817	18871
Reconnaissance	0.9239	0.8026	0.8590	3496
Shellcode	0.4043	0.8042	0.5381	378
Worms	0.3415	0.6364	0.4444	44

Table 4.2: Classification report of the final XGBoost Level 2 model

XGBoost performs best on frequent classes such as Generic, Fuzzers, Reconnaissance, and Exploits. Rare classes, especially Analysis and Backdoor, remain difficult to classify, which explains the lower macro F1-score compared to the weighted F1-score.

4.3.4 Confusion Matrix Analysis

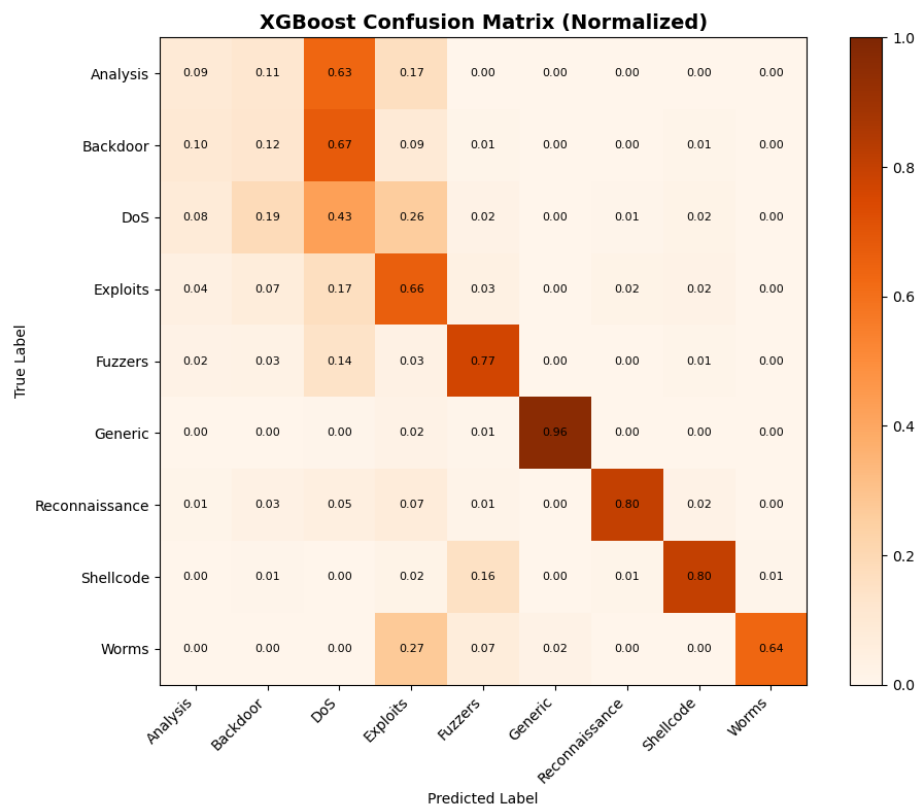


Figure 4.6: Normalized confusion matrix of the final XGBoost Level 2 model

The normalized confusion matrix shows that XGBoost does a good job with the main attack types, especially Generic, Reconnaissance, Shellcode, Fuzzers, Exploits, and Worms. Generic has the best result, with 96% correctly classified, and both Reconnaissance and Shellcode reach 80%.

However, the model has trouble with less common classes such as Analysis and Backdoor, with only 9% and 12% classified correctly. Most of these samples are confused with DoS, which means these categories have similar traffic patterns.

Overall, the model works well for common classes but is less reliable for rare or overlapping ones. This explains the lower macro F1-score.

Chapter 5: Implementation on Raspberry Pi 4

5.1 Introduction

Previous chapters covered the design and offline evaluation of the two-level detection system. This chapter documents its operational deployment on a Raspberry Pi 4 as a hierarchical IDS server (`ids_cascade.py`) receiving real-time network traffic via TCP, alongside a client simulator (`client_simulator.py`) running on an external machine to inject flows from the test dataset.

5.2 Environment Setup on Raspberry Pi

5.2.1 Hardware Specifications

The Raspberry Pi 4 Model B used for this deployment features a quad-core ARM Cortex-A72 processor at 1.5 GHz and 4 GB of LPDDR4 RAM, running on a 64-bit Raspberry Pi OS. The installed Python dependencies were strictly limited to those necessary for inference: `scikit-learn`, `xgboost`, `joblib`, `pandas`, and `numpy`.

5.2.2 Deployed Artifacts

The artifacts are organized into two distinct directories on the device to maintain modularity:

Directory: `level1_files/`

File	Content
<code>ids_model.joblib</code>	Random Forest Level 1 model
<code>ids_mapping.joblib</code>	Categorical features mapping
<code>ids_columns.joblib</code>	Ordered list of expected columns
<code>ids_cat_features.joblib</code>	List of categorical features

Directory: `level2_files/`

File	Content
<code>xgb_level2_model.pkl</code>	XGBoost Level 2 model
<code>preprocessor_level2.pkl</code>	Complete ColumnTransformer pipeline
<code>attack_label_encoder.pkl</code>	Decoding from indices to attack names
<code>level2_metadata.json</code>	Rare category grouping parameters

This separation accurately reflects the two-level architecture: Level 1 for binary filtering, and Level 2 for attack category classification.

5.3 Deploying the Selected Models

5.3.1 Client/Server Architecture

The architecture relies on a standard TCP communication model. The Raspberry Pi continuously listens on port 5000 (interface 0.0.0.0). An external machine acts as the client, sending JSON-formatted network flows via TCP. The `SO_REUSEADDR` option is enabled on the server socket to allow immediate restarts without waiting for the TCP protocol's `TIME_WAIT` delay to expire.

5.3.2 The Hierarchical IDS Server

At startup, all artifacts for both levels are loaded into memory once using `joblib.load()`. The `level2_metadata.json` file is also parsed to retrieve `common_categories`, which is necessary for the "rare category grouping" process in Level 2.

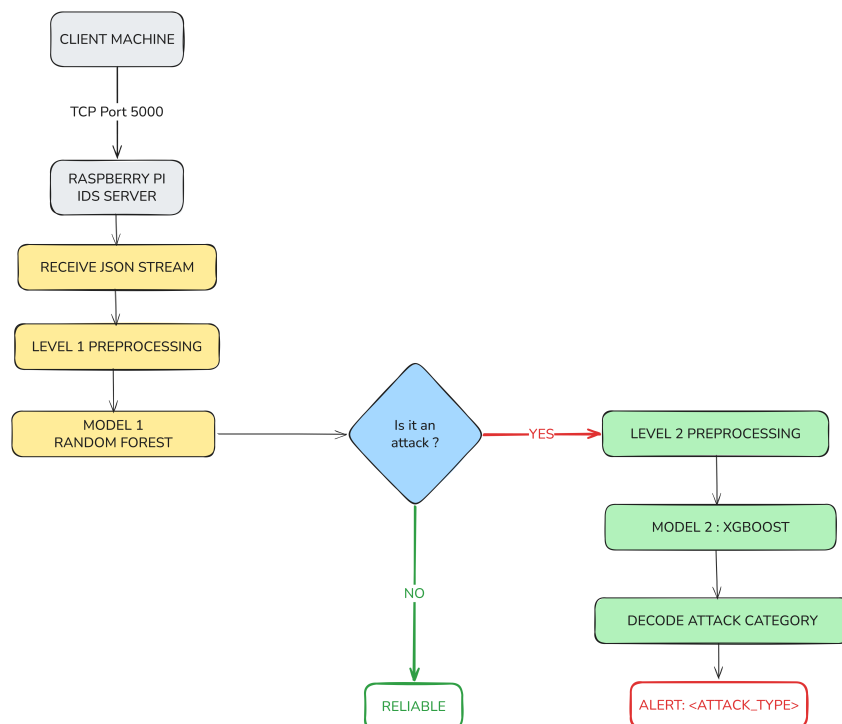


Figure 5.1: Hierarchical IDS Deployment Architecture and Decision Pipeline

For each incoming connection, the processing pipeline operates as follows:

- **Level 1 - Binary Filtering:**
 - The received JSON is deserialized into a single-row `pd.DataFrame`.
 - Categorical features (`proto`, `service`, `state`) are encoded via `ids_mapping.joblib`, with unknown values replaced by `-1`.
 - Columns are reordered according to `ids_columns.joblib`.
 - The Random Forest model produces a binary prediction.
 - If the verdict is 0 → the system displays **RELIABLE TRAFFIC**, and processing ends for this packet.

- **Level 2 - Attack Classification (if verdict == 1):**
 - Rare category grouping is manually applied: any categorical value not present in `common_categories` is replaced by "OTHER".
 - The `preprocessor_level2` (ColumnTransformer handling imputation, scaling, and one-hot encoding) transforms the flow.
 - The XGBoost model predicts the class index, which is decoded into a specific attack name via `le_l2.inverse_transform()`.
 - The confidence score is calculated using `predict_proba().max() * 100`.
 - The system displays: **ALERT: <ATTACK_NAME> detected (XX.XX%)**.

Error handling via `try/except/finally` blocks ensures that the connection is properly closed and the server maintains continuity even in the event of malformed flows.

5.3.3 The Client Simulator

The client randomly samples `nb_paquets` rows (20 by default) from the `UNSW_NB15_testing-set.csv` file. Each row is serialized into JSON and sent over an independent TCP connection, with a one-second delay between transmissions to simulate a steady, progressive flow. The server IP is configurable, allowing easy switching between local testing (127.0.0.1) and real deployment (the Raspberry Pi's IP address).

5.4 Testing and Performance Evaluation

5.4.1 Testing Protocol

Tests were conducted under two configurations: a local setup (client and server running on the same machine) to validate the software pipeline, and a distributed setup (client on a PC, server on the Raspberry Pi) to evaluate performance under real-world conditions. In both scenarios, 20 flows were sequentially injected over a timeframe of approximately 20 seconds.

5.4.2 Observed Outputs

A typical session produces the following output on the Raspberry Pi terminal:

```
Loading Hierarchical IDS brains...
HYBRID IDS active on port 5000. Waiting for traffic...
[192.168.1.42] RELIABLE TRAFFIC
[192.168.1.42] ALERT : EXPLOITS detected (91.34%)
[192.168.1.42] RELIABLE TRAFFIC
[192.168.1.42] ALERT : DOS detected (87.12%)
```

The displayed attack category and confidence score provide immediate, actionable intelligence that an operator can use to prioritize security alerts.

5.4.3 Latency and Throughput

The inference latency per flow is on the order of a few milliseconds on the Raspberry Pi, which is well below the one-second delay imposed by the simulator. The current bottleneck is not the machine learning model, but rather the sequential handling of TCP connections, a connection must be entirely processed before the next one is accepted. In this low-throughput testing scenario, this limitation has no practical impact.

5.5 Global Performance of the Two-Level Approach

5.5.1 Efficiency of the Hierarchical Pipeline

The cascade architecture offers a concrete advantage on embedded hardware: normal network flows, which constitute the vast majority of real-world traffic, are filtered out at Level 1. Consequently, the heavier XGBoost pipeline is never solicited for benign traffic. Only flows flagged as potential attacks pass through to Level 2, keeping the average CPU load significantly lower than theoretical worst-case scenarios.

5.5.2 Train/Inference Consistency

A critical deployment requirement is ensuring that the transformations applied during live inference strictly match those used during training. The rare category grouping is manually reproduced using the `common_categories` retrieved from `level2_metadata.json`. Furthermore, the serialized `preprocessor_level2` applies the exact same scaling, and one-hot encoding logic. This rigorous consistency guarantees that the model evaluates data within the exact feature distribution for which it was optimized.

5.5.3 Limitations

The sequential connection handling currently implemented would become a bottleneck under heavy, sustained traffic. Implementing multi-threading (e.g., assigning a `threading.Thread` per connection) would allow the system to leverage the four cores of the Raspberry Pi and process multiple flows in parallel. Additionally, the receive buffer is currently fixed at 8192 bytes; while this easily accommodates a single UNSW-NB15 JSON flow, proper TCP fragmentation management would be required for a production-grade deployment.

Chapter 6: Federated Learning

6.1 Introduction

This chapter presents the integration of Federated Learning (FL) within our IDS architecture. This paradigm shifts from a localized training model on a single probe to a distributed collective intelligence. By utilizing the Flower framework, we have transformed our IDS nodes into clients capable of collaborating to improve Level 1 binary detection without ever exchanging raw data.

6.2 Motivation for Federated Learning in Intrusion Detection

The adoption of Federated Learning is driven by three critical factors:

- **Data Privacy:** Network logs contain sensitive information that never leaves the local Raspberry Pi. Only mathematical parameters (weights and biases) are transmitted.
- **Collective Intelligence:** An isolated probe may not encounter certain types of attacks. FL allows the system to benefit from the experience of all network nodes to strengthen the global model.
- **Network Efficiency:** Transferring model parameters is significantly less bandwidth-intensive than the massive exportation of PCAP files to a centralized server.

6.3 Federated Learning Architecture

6.3.1 Centralized vs Federated Learning

In a centralized system, all data from the UNSW-NB15 dataset would be aggregated on a single server. In our federated approach, data is partitioned into three distinct silos (clients). The server only sees the individual model's parameters.

6.3.2 Client–Server Communication Model

The model relies on iterative communication managed by the server via the gRPC protocol:

- **Broadcast:** The server sends the initial model (MLPClassifier) to the clients.
- **Local Training:** Each client trains the model on its local data partition.
- **Update:** Clients send their updated weights and biases back to the server.
- **Aggregation:** The server merges these parameters to create an updated global model.

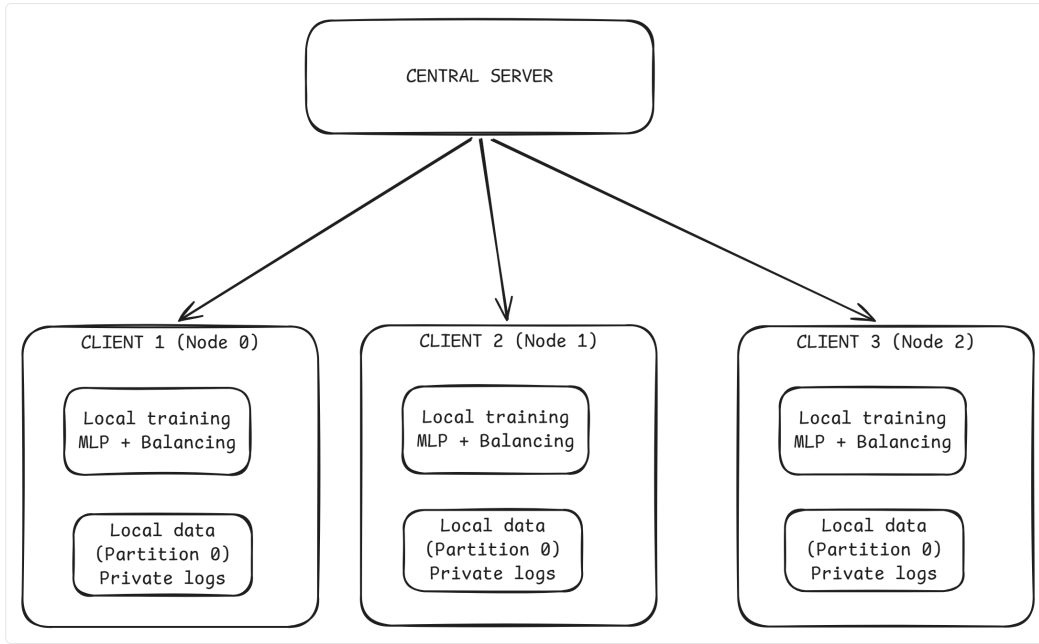


Figure 6.1: Federated Learning Architecture and Communication Flow

6.4 Federated Training Strategy for Level 1

6.4.1 Local Model Training

The local model used is an `MLPClassifier` with two hidden layers of 10 neurons each. At each federated round, each client receives the global model parameters from the server and trains it on its local data partition.

6.4.2 Model Aggregation Process

The server employs the **FedAvg** (Federated Averaging) strategy. Aggregation involves calculating the arithmetic mean of the weights (W) and biases (b) received from the clients:

$$W_{global} = \frac{1}{n} \sum_{i=1}^n W_i, \quad b_{global} = \frac{1}{n} \sum_{i=1}^n b_i \quad (6.1)$$

This process is repeated over 5 rounds.

6.4.3 Integration with the Level 1 Classifier

Once the rounds are completed, the final global model is saved as `federated_ids_model.joblib`. This file replaces the standard binary classifier as the first shield of our hierarchical IDS, benefiting from enhanced generalization capabilities.

6.5 Discussion and Limitations

While effective, this approach has limitations:

- **Data Heterogeneity:** If clients have highly different traffic distributions (Non-IID), convergence may be slower.
- **RAM Consumption:** Although the MLP is lightweight, training on a Raspberry Pi is more resource-intensive than simple inference, requiring strict resource management.

6.6 Conclusion

Federated Learning transforms the IDS from a passive probe into a collaborative, privacy-respecting system. By combining the Flower framework with our two-level architecture, we have demonstrated that it is possible to achieve robust detection performance (87% recall for normal traffic) while keeping local data entirely secure.

Chapter 7: General Conclusion

7.1 Summary of Achievements

This project successfully implemented a Network Intrusion Detection System based on a two-level machine learning architecture. The first level performs binary classification to distinguish between normal and malicious traffic, while the second level classifies detected attacks into specific attack categories. The system was trained and evaluated using the UNSW-NB15 dataset, then deployed on a Raspberry Pi 4 to validate its feasibility on embedded hardware.

In addition, Federated Learning was integrated using the Flower framework to improve the Level 1 detection model without sharing raw data between clients. This approach showed how multiple IDS nodes can collaborate while preserving data privacy. Overall, the project demonstrates that machine learning-based intrusion detection can be deployed in a lightweight and distributed way.

7.2 Challenges Encountered

Several challenges were encountered during the project. The first challenge concerned data pre-processing. Another important challenge was the class imbalance in the UNSW-NB15 dataset, which affected the detection of minority attack classes and required balancing techniques.

Deployment on Raspberry Pi introduced technical constraints: limited computational resources, memory constraints, and the need for fast, stable inference. In Federated Learning, ensuring server-client communication and managing model aggregation and training consistency were challenging.

7.3 Limitations and Improvement Perspectives

While the system demonstrates significant potential, certain limitations persist. The TCP server processes connections one at a time, potentially leading to bottlenecks during periods of high network traffic. Implementing multi-threading or asynchronous processing could alleviate this issue. Additionally, the system currently utilizes simulated flows rather than live network traffic. Incorporating real-time data extraction would make the Deployment is more realistic.

Classifying rare attack categories remains challenging due to insufficient representation in the dataset. Future work could focus on enhancing minority-class detection and expanding Federated Learning experiments with additional clients, non-IID data distributions, and prolonged training rounds.

In summary, the project successfully accomplished its primary objective by designing, training, deploying, and evaluating a hierarchical intrusion detection system on the Raspberry Pi 4 platform, alongside an initial Federated Learning extension to facilitate privacy-preserving collaboration.

Bibliography

- [1] Nour Moustafa and Jill Slay. UNSW-NB15: A comprehensive data set for network intrusion detection systems. In *Proceedings of the Military Communications and Information Systems Conference (MilCIS)*. IEEE, 2015.
- [2] UNSW Canberra Cyber. The UNSW-NB15 dataset. <https://research.unsw.edu.au/projects/unsw-nb15-dataset>, 2015. Accessed: 2026-05-10.
- [3] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM, 2016.
- [4] XGBoost Developers. XGBoost documentation. <https://xgboost.readthedocs.io/>. Accessed: 2026-05-10.
- [5] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [6] imbalanced-learn Developers. SMOTETomek. <https://imbalanced-learn.org/stable/references/generated/imblearn.combine.SMOTETomek.html>. Accessed: 2026-05-10.
- [7] imbalanced-learn Developers. SMOTENC. https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTENC.html. Accessed: 2026-05-10.
- [8] scikit-learn Developers. Metrics and scoring: Quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html. Accessed: 2026-05-10.
- [9] Flower Labs. What is federated learning? <https://flower.ai/docs/framework/tutorial-series-what-is-federated-learning.html>, 2023. Accessed: 2026-05-10.
- [10] Flower Labs. Get started with flower (PyTorch). <https://flower.ai/docs/framework/tutorial-series-get-started-with-flower-pytorch.html>, 2023. Accessed: 2026-05-10.
- [11] Flower Labs. Flower documentation. <https://flower.ai/docs/>. Accessed: 2026-05-10.
- [12] Joblib Developers. Joblib: Running Python functions as pipeline jobs. <https://joblib.readthedocs.io/en/stable/>. Accessed: 2026-05-10.
- [13] Matplotlib Developers. Matplotlib documentation. <https://matplotlib.org/stable/index.html>. Accessed: 2026-05-10.
- [14] NumPy Developers. NumPy documentation. <https://numpy.org/devdocs/>. Accessed: 2026-05-10.
- [15] pandas Developers. pandas documentation. <https://pandas.pydata.org/docs/>. Accessed: 2026-05-10.
- [16] scikit-learn Developers. scikit-learn user guide. https://scikit-learn.org/stable/user_guide.html. Accessed: 2026-05-10.